



Software Factory en la Era de la IA

Cultura Equipo

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Cultura Ingenieria

Definición

La cultura de ingeniería es el conjunto de valores, prácticas y normas compartidas que determinan cómo un equipo construye software. No es solo "cómo hacemos las cosas", sino por qué las hacemos de esa manera.

Modelo de Westrum

Ron Westrum clasificó las culturas organizacionales en tres tipos:

Tipo	Características	Resultado
Pathological	Miedo, información ocultada, competencia tóxica	Silos, bajo desempeño
Bureaucratic	Procesos rígidos, reglas sobre resultados	Lentitud, mediocridad
Generativa	Confianza, información fluye, aprendizaje continuo	Alto desempeño, innovación

Señales de cada tipo

Pathological:

- Bugs se ocultan, no se reportan incidentes
- Blame culture después de fallos
- Knowledge hoarding como poder personal

Bureaucratic:

- Procesos de approval que toman semanas
- Documentación que nadie lee
- "Eso no es mi responsabilidad"

Generativa:

- Post-mortems sin blame, enfocados en mejora
- Documentación viva y actualizada
- Cross-functional collaboration natural

Cómo Construir Cultura de Ingeniería

1. Definir Valores Explícitos

No asumir que la cultura "ocurre". Definirla y comunicarla.

Ejemplo de Engineering Culture Playbook:

VALOR: "Ship early, ship often"
– Preferimos deploys pequeños y frecuentes

- Feature flags > long-lived branches
- Medimos deployment frequency como health check

VALOR: "Blameless post-mortems"

- Enfocados en sistema, no en personas
- Acciones correctivas con due dates
- Compartidos públicamente en la org

VALOR: "Documentation as code"

- READMEs actualizados como Definition of Done
- ADRs para decisiones arquitectónicas
- Runbooks para operaciones

2. Liderazgo por Ejemplo

Acción del liderazgo	Impacto cultural
CTO escribe código y hace PRs	Engineering remains hands-on
Managers participan en on-call	Empatía con la experiencia del equipo
Leaders admiten errores públicamente	Seguridad psicológica crece
Leadership invierte en tooling	Productivity y developer experience importan

3. Crear Rituals que Refuercen la Cultura

Ritual	Frecuencia	Objetivo cultural
Tech Talks	Semanal/quincenal	Knowledge sharing
Demo Day	Fin de sprint	Pride en el trabajo
Blameless Post-mortem	Cada incidente	Learning, no blame
Architecture Review	Mensual	Quality, standards
Hackathon	Trimestral	Innovation, creativity

4. Codificar en Procesos

La cultura se sostiene en procesos, no en documentos:

- **Code review guidelines** refuerzan quality standards
- **Incident response process** refuerza blameless culture
- **Hiring rubric** refuerza cultural alignment
- **Onboarding checklist** transmite cultura a nuevos miembros

Frameworks de Cultura de Ingeniería

Spotify Model (Squad, Tribe, Chapter, Guild)

Unidad	Propósito	Cultural impact
Squad	Equipo autónomo cross-functional	Ownership, autonomía
Chapter	Grupo de mismo skill set	Best practices, mentoring
Guild	Comunidad de interés transversal	Knowledge sharing
Tribe	Agrupación de squads alineados	Alignment, escala

Netflix Culture Deck

Principios clave:

- **Freedom & Responsibility:** alta autonomía con alta responsabilidad
- **Context, not Control:** dar contexto en lugar de microgestionar
- **Highly Aligned, Loosely Coupled:** alineados en dirección, independientes en ejecución

Team Topologies (Team Types)

Tipo	Responsabilidad	Cultural impact
Stream-aligned	Feature delivery	Ownership end-to-end
Platform	Internal developer platform	Self-service, developer experience
Enabling	Upskill other teams	Knowledge transfer, mentoring
Complicated-subsystem	Complex technical domain	Deep expertise

Métricas de Salud Cultural

Métrica	Cómo medir	Target
eNPS	Survey trimestral	> 30
Psychological Safety Score	Survey con escala Likert	> 4.0/5
Knowledge Sharing Index	Tech talks, docs contributions	Creciente
Retention Rate	HR data	> 85% anual
Internal Mobility	Transferencias internas	> 10% anual

Anti-Patterns de Cultura

Anti-Patrón	Consecuencia
"Culture fit" como excusa para homogeneidad	Lack de diversidad

Anti-Patrón	Consecuencia
Culture as perks (ping pong, beer fridge)	Superficial, no duradera
Hero culture	Bus factor = 1, burnout
"Move fast and break things" sin quality bar	Technical debt 累积
Perfectionism culture	Paralysis, missed deadlines

Conexiones

- Seguridad Psicológica — base de la cultura generativa
- Compartir Conocimiento — ritual cultural clave
- Communities of Practice — cómo escalar cultura
- Salud del Equipo — medir lo que importa
- DevOps Filosofía — cultura como pilar DevOps
- Agile Manifiesto — valores que inspiran
- OKRs — alinear cultura con objetivos

Psychological Safety

Definición

La seguridad psicológica es la creencia compartida de que el equipo es seguro para tomar riesgos interpersonales. Conceptualizada por Amy Edmondson (Harvard) y validada por Google Project Aristotle.

"Psychological safety is not about being nice. It's about giving candid feedback, openly admitting mistakes, and learning from each other." — Amy Edmondson

Google Project Aristotle

Google estudió 180+ equipos para determinar qué factores predicen la efectividad del equipo.

Los 5 factores (en orden de impacto)

#	Factor	Descripción
1	Psychological Safety	Puedo ser vulnerable sin consecuencias negativas
2	Dependability	Mis compañeros entregan trabajo de calidad a tiempo
3	Structure & Clarity	Roles, planes y objetivos son claros
4	Meaning	Mi trabajo tiene propósito personal
5	Impact	Creo que mi trabajo importa y crea cambio

Hallazgo clave: La seguridad psicológica fue el predictor #1 — más que la composición del equipo, la seniority, o la mezcla de skills.

Los 4 niveles de Seguridad Psicológica

Nivel	Descripción	Ejemplo
Inclusion Safety	Siento que pertenezco al grupo	"Mi opinión es bienvenida"
Learner Safety	Puedo preguntar y aprender sin vergüenza	"No sé X, puedo aprender"
Contributor Safety	Puedo contribuir ideas sin miedo	"Propongo esta solución"
Challenger Safety	Puedo cuestionar el status quo	"Creo que我们应该 hacer Y en lugar de X"

Cómo Construir Seguridad Psicológica

Para Líderes

Acción	Ejemplo concreto
Modelar vulnerabilidad	"No sé la respuesta, investiguemos juntos"
Invitar participación explícitamente	"Quiero escuchar a los que no han hablado"
Responder bien a errores	"¿Qué aprendimos de este incidente?"
Dar feedback constructivo	"El approach tiene mérito, veamos alternativas para X"
Celebrar learning	Compartir propios errores y aprendizajes

Para Equipos

Práctica	Frecuencia	Impacto
Start/Stop/Continue	Fin de sprint	Feedback abierto
Blameless Post-mortems	Cada incidente	Aprendizaje sin miedo
Pre-mortem	Inicio de proyecto	Identificar riesgos temprano
Retrospective Safety Check	Cada retro	Verificar que el espacio es seguro
"I don't know" Culture	Diario	Reducir pretensión

Señales de Falta de Seguridad Psicológica

Señal	Qué significa
Nadie cuestiona decisiones del tech lead	Fear-based compliance
Errores se reportan tarde o no se reportan	Blame culture
Solo hablan los mismos 2-3 personas	Exclusion, fear
Code reviews superficiales	"Don't want to rock the boat"
High bus factor en módulos críticos	Knowledge hoarding
Retros vacías o superficiales	No trust in process

Encuesta de Seguridad Psicológica

Escala Likert 1-5 (1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo):

#	Pregunta
1	Si cometo un error en este equipo, se usa contra mí
2	Los miembros del equipo pueden presentar problemas y malas noticias
3	Es seguro correr un riesgo en este equipo

#	Pregunta
4	Es difícil pedir ayuda a otros miembros del equipo
5	Nadie en el equipo deliberadamente sabotea mis esfuerzos
6	Mis habilidades únicas son valoradas y utilizadas
7	Aporto ideas innovadoras que son bien recibidas
8	Puedo expresar mi opinión sin miedo a represalias
9	Mi opinión cuenta tanto como la de otros
10	Siento que pertenezco a este equipo

Scoring:

- Promedio > 4.0: Saludable
- Promedio 3.0-4.0: Necesita atención
- Promedio < 3.0: Crítico — intervención necesaria

Seguridad Psicológica y Desempeño

Métrica de equipo	Impacto de alta safety
Innovation rate	+27% (Harvard Business Review)
Learning from failure	+40% más willingness to experiment
Employee retention	+18% lower turnover
Team performance	Top predictor (Project Aristotle)
Error reporting	Más temprano = más barato de arreglar

Anti-Patterns

Anti-Patrón	Consecuencia
Confundir safety con confort	"Todos deben estar de acuerdo" = groupthink
Safety sin accountability	"No importa si fallas" = low standards
Safety selectiva	Solo para ciertos miembros = exclusion
Forzar vulnerability	"Compártelo todo" = invasión de privacidad

Conexiones

- Cultura de Ingeniería — safety es base de la cultura generativa
- Burnout — falta de safety contribuye a burnout

- Salud del Equipo — medir safety como métrica
- Estimación — safety permite estimaciones honestas
- DevOps — blameless culture como pilar
- Mentoring — safety enables learning
- Diversidad e Inclusión — inclusion requiere safety

Remote Hybrid

Modelos de Trabajo

Modelo	Descripción	Trade-offs
Fully Remote	Sin oficina, talent global	Flexibilidad total, timezone challenges
Hybrid (structured)	Días fijos oficina/remoto	Predictable collaboration, commuting
Hybrid (flexible)	Employee choose days	Autonomía, coordination harder
Remote-first	Oficina opcional, async by default	Best of both, requires discipline
Office-first	Remoto como excepción	Culture osmosis, limited talent pool

Async Communication como Default

El trabajo remoto eficiente se basa en comunicación asíncrona.

Jerarquía de canales por urgencia

Urgencia	Canal	Ejemplo
Crítica (minutos)	Phone call, Slack DM urgente	Production down, P0 incident
Alta (horas)	Slack message con @mention	Blocker que necesita respuesta rápida
Normal (día)	Slack thread, Notion comment	Question no blocking
Baja (semana)	Email, document comment	FYI, non-urgent updates
Background	Recorded video, Loom	Demo, walkthrough, context-sharing

Reglas de Async Communication

- Assume positive intent** — Sin tono vocal, los mensajes se interpretan peor
- Write for scannability** — Headers, bullets, bold key points
- Provide context** — No asumas que el lector tiene el mismo background
- Use threads** — Mantener conversaciones organizadas
- Set response expectations** — "Necesito esto para EOD" vs "Cuando puedas"
- Record decisions** — No confíes en la memoria colectiva

Timezone Management

Estrategia	Descripción	Cuándo usar
Overlap hours	4-6h de overlap entre timezones	Equipos en 2-3 zonas

Estrategia	Descripción	Cuándo usar
Follow-the-sun	Handoff entre zonas	Soporte 24/7, CI/CD
Async-first	Trabajo independiente de timezone	Equipos globales 4+ zonas
Core hours	Bloque de horas comunes para meetings	Reuniones síncronas necesarias

Prácticas para timezones

- **Recordar todas las meetings** — Los que no asisten no quedan out
- **Meeting notes en vivo** — Quien async puede contribuir después
- **Decision docs** — Las decisiones se documentan, no se "dicen en la call"
- **Rotate meeting times** — Evitar que siempre los mismos se adapten

Herramientas para Trabajo Remoto

Categoría	Herramientas	Caso de uso
Async video	Loom, Vidyard, Screen Studio	Demos, walkthroughs
Async text	Notion, Coda, Confluence	Documentación, decisiones
Real-time chat	Slack, Teams, Discord	Comunicación diaria
Video calls	Zoom, Google Meet, Gather	Meetings síncronas
Virtual office	Gather, Teamflow, Sococo	Watercooler, serendipity
Project management	Linear, Jira, Asana	Tracking de trabajo

Challenges del Trabajo Remoto

Challenge	Solución
Isolation / Loneliness	Virtual coffee, pair programming, coworking sessions
Communication overhead	Async-first, reduce meetings, better documentation
Blurred boundaries	No-work hours, "commute" ritual, clear expectations
Onboarding difficulty	Buddy system, structured 30-60-90 plan
Performance visibility	Output-based evaluation, regular demos
Timezone friction	Async-first, overlap hours, recorded meetings

Design de Oficina Híbrida

Si se mantiene oficina, diseñarla para collaboration, no para individual work:

Zona	Propósito	% del espacio
Collaboration zones	Whiteboards, pair programming	40%
Meeting rooms	Hybrid meetings (good AV)	30%
Quiet zones	Deep work, focus time	20%
Social zones	Watercooler, coffee	10%

Hybrid Meeting Best Practices

- Cada persona en su own screen (aunque estén en la misma sala)
- Use shared digital whiteboard (Miro, FigJam)
- Always-on screen showing remote participants
- "Remote-first" mindset: remote voices have equal weight

Métricas de Efectividad Remota

Métrica	Target	Cómo medir
Async response time	< 4h para normal, < 1h para high	Slack analytics
Meeting load	< 20h/semana por persona	Calendar analysis
Documentation coverage	> 90% decisions documented	Wiki metrics
Employee satisfaction	> 4.0/5 en surveys	Pulse surveys
Retention rate	Comparable to on-site	HR data

Anti-Patterns

Anti-Patrón	Consecuencia
"Always-on" culture	Burnout, expectations unclear
Surveillance software	Trust erosion, attrition
Meetings as default	Async regression, time waste
"Remote-friendly" vs "remote-first"	Second-class citizens for remote workers
No overlap hours	Complete isolation between timezones

Conexiones

- Herramientas de Colaboración — stack de herramientas detallado
- Burnout — remote work y burnout relationship
- Kanban — adaptar flujos a async
- Planificación Ágil — ceremonies en remoto

- DevOps — distributed teams y DevOps
- Seguridad Psicológica — remoto requiere más esfuerzo en trust
- Motivación — remote workers y motivation

Collaboration Tools

Stack de Herramientas por Categoría

Comunicación

Herramienta	Tipo	Mejor para	Costo
Slack	Chat async/sync	Teams técnicos, integraciones	\$
Microsoft Teams	Chat + meetings	Orgs que ya usan M365	\$\$
Discord	Chat + voice	Comunidades, startups	Free/\$
Zulip	Chat con topics	Async-heavy teams	Free/OSS

Documentación y Knowledge Management

Herramienta	Tipo	Mejor para	Costo
Notion	Wiki + database	Docs, project tracking, wikis	\$
Confluence	Enterprise wiki	Large orgs, Jira integration	\$\$
Linear docs	Embedded docs	Product-eng alignment	\$
Outline	Open-source wiki	Self-hosted knowledge base	Free/OSS
Obsidian	Local markdown	Personal knowledge, PKM	Free/\$

Project Management

Herramienta	Tipo	Mejor para	Costo
Linear	Issue tracking modern	Eng teams, keyboard-first	\$
Jira	Enterprise PM	Large orgs, complex workflows	\$\$
GitHub Projects	GitHub-native	Teams already on GitHub	Free
Shortcut	Balanced PM	Mid-size teams	\$
Height	AI-first PM	Fast-moving teams	\$

Visual Collaboration

Herramienta	Tipo	Mejor para	Costo
Miro	Whiteboard digital	Workshops, brainstorming	\$
FigJam	Design whiteboard	Design-eng collaboration	\$
Excalidraw	Simple sketches	Quick diagrams, OSS	Free
Mermaid	Diagram-as-code	Architecture, flowcharts	Free

Async vs Sync: Decision Framework

Situación	Async	Sync
Share status update	Written update	✗
Brainstorm ideas	⚠️ Possible but slower	Better with discussion
Resolve conflict	✗ Too nuanced	Needs dialogue
Code review	PR comments	⚠️ Pair review for complex
Decision making	⚠️ RFC + comments	Final sync for alignment
Onboarding	⚠️ Docs + async Q&A	Live sessions needed
Retrospective	✗ Needs psychological safety	Real-time feedback
Incident response	⚠️ Initial triage	War room for resolution

Selection Criteria

Evaluar herramientas con esta matriz:

Criterio	Peso	Questions to ask
Adoption friction	25%	¿El equipo lo va a usar?
Integration depth	20%	¿Se conecta con nuestro stack?
Async support	20%	¿Soporta working across timezones?
Cost	15%	¿Cabe en el budget?
Security/compliance	15%	¿Cumple requirements legales?
Vendor lock-in	5%	¿Podemos migrar si es necesario?

Setup Recommendations por Team Size

Size	Stack mínimo	Stack ideal
2-5	GitHub + Slack + Notion	+ Linear + Loom

Size	Stack mínimo	Stack ideal
6-15	+ Linear + Miro	+ Excalidraw + Gather
16-50	+ Confluence + Jira	+ Dedicated tooling team
50+	Enterprise suite	Platform team managing tooling

Integration Patterns



Anti-Patterns

Anti-Patrón	Consecuencia
Tool sprawl (10+ tools)	Context switching, confusion
"This tool will fix our culture"	Tools enable, don't create culture
No integration between tools	Information silos
Choosing tools by hype	Adoption failure
No training/onboarding on tools	Low adoption, frustration

Conexiones

- Trabajo Remoto — tools para distributed teams
- Compartir Conocimiento — tools como enablement
- Herramientas — stack de herramientas del proyecto
- Prácticas — prácticas que estas herramientas habilitan
- Cultura de Ingeniería — tools refuerzan cultura
- Ritmos de Delivery — tools soportan ceremonies
- CI/CD — integración con pipeline

Knowledge Sharing

Por qué Compartir Conocimiento

Costo de no compartir	Impacto
Knowledge silos	Bus factor = 1, single point of failure
Duplicate effort	Mismo problema resuelto 3 veces
Slow onboarding	New hires tardan meses en ser productivos
Tribal knowledge	Información crítica vive solo en cabezas
Repeated mistakes	Aprendizaje no escala

Formatos de Knowledge Sharing

Tech Talks / Lunch & Learns

Aspecto	Best Practice
Frecuencia	Semanal o quincenal
Duración	30-45 min + 15 min Q&A
Formato	20 min charla + demo + discussion
Audience	Voluntario, anyone can attend
Recording	Siempre grabar para async access
Topics	Technical deep-dives, post-mortems, tool demos

Tech Talk Template

[Título]
Problem Statement ¿Qué problema resolvemos?
Context ¿Qué estábamos haciendo antes?
Solution ¿Qué hicimos? Demo o walkthrough.
Results

¿Qué impacto tuvo? Métricas si available.

Learnings

¿Qué aprenderíamos differently?

Brown Bag Sessions

- Informales, durante lunch
- Temas más broad que deep dives
- Buenos para onboarding y cross-training
- Menos pressure que tech talks formales

Guilds y Chapters (Spotify Model)

Unidad	Definición	Actividad
Chapter	Personas con el mismo skill set (dentro de tribu)	Best practices, 1:1s, hiring
Guild	Comunidad de interés transversal (cross-tribe)	Knowledge sharing, talks, tools

Documentation Culture

La documentación es knowledge sharing asíncrono y duradero.

Tipo de doc	Audiencia	Formato
Architecture Decision Record (ADR)	Engineering	Markdown con contexto, options, decision
Runbook	On-call engineers	Step-by-step procedures
RFC / Design Doc	Stakeholders	Propuesta + discussion
How-to guide	Developers	Tutorial-style, practical
Post-mortem	Whole org	Incidente, timeline, learnings

ADR Template

ADR-001: Use PostgreSQL as primary database

Status: Accepted

Date: 2026-07-13

Context

Need a relational database for the new payment service.

Decision

We will use PostgreSQL 16.

Consequences

- Positive: ACID, mature ecosystem, JSON support
- Negative: Need to learn PostgreSQL-specific features
- Neutral: Team has mixed experience (2/5 expert, 3/5 basic)

Programs Estructurados

Learning Circles

- Grupo de 4-6 personas que se reúnen semanalmente
- Leen/artículo/book juntos, discuten, aplican
- Rotate facilitator each session
- Low commitment, high learning

Internal Conference

Fase	Actividad
Call for papers	2-4 weeks before
Schedule	1-day event, 4-5 tracks
Presentations	20-30 min each
Workshops	60-90 min hands-on
Hackathon	Post-conference, 1 day

Reading Groups

- Book club technical (ej: "Designing Data-Intensive Applications")
- Article of the week
- Paper reading (para equipos más advanced)
- Shared notes and discussion

Métricas de Knowledge Sharing

Métrica	Target	Cómo medir
Tech talks / month	> 4	Calendar tracking
Documentation coverage	> 80% de decisiones documented	Wiki audit
Onboarding time	< 4 weeks to first PR	HR + Git data
Bus factor	> 3 en módulos críticos	Code ownership analysis
Internal mobility	> 10% transfers/year	HR data

Incentives para Compartir

Incentivo	Implementación
Time allocation	Dedicate 10% del sprint para learning
Recognition	Highlight top contributors en all-hands
Career progression	Knowledge sharing como promotion criterion
Hackathons	Competitions que generan knowledge sharing
Blog interno	Platform para publicar learnings

Anti-Patterns

Anti-Patrón	Consecuencia
"Documentation is optional"	Knowledge silos
"Only seniors should present"	Excludes fresh perspectives
"We don't have time to share"	Short-term speed, long-term debt
"Just ask Bob"	Bus factor = 1
Recording without follow-up	Content graveyard

Conexiones

- Cultura de Ingeniería — sharing culture es cultural pillar
- Communities of Practice — formalized sharing
- Mentoring — 1:1 knowledge transfer
- Documentation — documentation standards
- DevOps — learning organization
- Seguridad Psicológica — safety enables sharing
- Velocity — knowledge sharing improves velocity

Mentoria Coaching

Mentoring vs Coaching

Aspecto	Mentoring	Coaching
Enfoque	Career growth, wisdom transfer	Performance improvement, skill building
Relación	Senior → Junior	Peer o External
Duración	Long-term (6-18 months)	Short-medium term (3-6 months)
Agenda	Mentor-led (guidance)	Coachee-led (discovery)
Métrica	Career progression, retention	Behavior change, skill improvement

Tipos de Mentoring Programs

1:1 Mentoring (Traditional)

Aspecto	Best Practice
Matching	Shared interests, career goals, personality fit
Cadence	1 hora/quincenal, minimum
Duration	6-12 months con option to renew
Structure	Mentee drives agenda, mentor provides perspective
Documentation	Shared notes, goals tracking

Pair Mentoring

- Dos personas de nivel similar aprendiendo juntas
- Mutually beneficial: different strengths
- Less hierarchical, more collaborative
- Good for cross-functional learning

Reverse Mentoring

Beneficiario	Quién enseña	Tema típico
Senior leader	Junior developer	New tech, tools, market trends
Engineering manager	IC senior	Hands-on skills, technical depth
Established team	New hire	Fresh perspectives, outside experience

Impacto: Breaks hierarchy, builds empathy, accelerates learning bidirectionally.

Group Mentoring / Mentoring Circles

- 1 mentor con 3-5 mentees
- Peer learning + mentor guidance
- Scalable when 1:1 isn't feasible
- Topic-focused cohorts

Program Design Template

Phase 1: Setup (Month 1)

Activity	Owner	Timeline
Define goals & success metrics	HR + Eng Leadership	Week 1
Create mentee application form	Program manager	Week 2
Recruit mentors (voluntary)	Engineering managers	Week 2-3
Matching process	Program manager	Week 3-4
Kickoff event	All participants	Week 4

Phase 2: Active Mentoring (Months 2-8)

Activity	Cadence
1:1 meetings	Quincenal
Group check-ins	Mensual
Progress review	Trimestral
Mid-point survey	Month 4
Mentor training	Once + refreshers

Phase 3: Wrap-up (Month 9)

Activity	Propósito
Final reflection session	Closure, celebration
Outcome survey	Measure impact
Success stories	Internal marketing
Program iteration	Improve for next cohort

Conversation Frameworks

GROW Model (for Coaching)

Letter	Question	Purpose
Goal	"¿Qué quieres lograr?"	Clarity on objective
Reality	"¿Dónde estás ahora?"	Current state assessment
Options	"¿Qué podrías hacer?"	Generate alternatives
Will	"¿Qué vas a hacer?"	Commitment to action

Mentoring Conversation Starter

1. "¿Cuál es tu mayor challenge ahora mismo?"
2. "¿Qué skill quieres desarrollar este quarter?"
3. "¿Cómo te ves en 2-3 años?"
4. "¿Qué obstáculos ves para llegar ahí?"
5. "¿En qué te puedo ayudar específicamente?"

Anti-Patterns

Anti-Patrón	Consecuencia
Forced pairing (no chemistry)	Waste of time, resentment
Mentor without training	Bad advice, missed opportunities
No time allocation	"I don't have time" kills program
Only for underperformers	Stigma, low adoption
No structure or goals	Drift, no measurable impact

Conexiones

- Career Ladders — mentoring supports progression
- Compartir Conocimiento — mentoring as knowledge transfer
- Seguridad Psicológica — safety enables honest conversations
- Communities of Practice — communities as mentoring context
- Salud del Equipo — mentoring improves retention
- Agile Manifesto — sustainable pace includes growth
- Contratación — mentoring improves onboarding

Career Ladders

Dual Track: IC vs Management

Aspecto	Individual Contributor (IC)	Management
Impact	Through technical excellence	Through people and teams
Skills	Deep technical, system design	Leadership, strategy, people
Day-to-day	Code, design, review	1:1s, planning, hiring
Growth	Depth/breadth of expertise	Scope of responsibility
Compensation	Can reach Staff+ levels	Similar comp at director+

Why Dual Track Matters

- Not all great engineers want to manage
- Forcing IC into management loses technical talent
- Clear IC path retains top individual contributors
- Both tracks should have equal prestige and compensation

Typical Leveling Framework

IC Track

Level	Title	Scope	Impact
L3	Junior Engineer	Individual tasks with guidance	Learn and contribute
L4	Software Engineer	Independent features	Own deliverables
L5	Senior Engineer	Multiple features / subsystems	Team-level impact
L6	Staff Engineer	Cross-team technical direction	Org-level impact
L7	Principal Engineer	Company-wide technical strategy	Company-level impact
L8	Distinguished Engineer	Industry-level influence	Industry-level impact

Management Track

Level	Title	Scope	Reports
M1	Engineering Manager	1 team (5-8 engineers)	Direct reports
M2	Senior EM / Director	Multiple teams	EMs + ICs

Level	Title	Scope	Reports
M3	VP Engineering	Department	Directors + EMs
M4	CTO / SVP Engineering	Company-wide	VPs + Directors

Promotion Criteria Matrix

IC Promotion: Senior (L4 → L5)

Dimension	Expectation	Evidence
Technical	Design features end-to-end	ADRs, system design docs
Execution	Delivers complex features independently	PRs, shipped features
Impact	Improves team practices	Process improvements, tool adoption
Communication	Clear written and verbal	Design docs, tech talks
Mentoring	Helps junior engineers grow	Code reviews, mentoring sessions

IC Promotion: Staff (L5 → L6)

Dimension	Expectation	Evidence
Technical	Sets technical direction for team/org	Tech strategy docs, architecture
Execution	Delivers ambiguous, large-scale projects	Multi-quarter projects
Impact	Influences multiple teams	Cross-team initiatives
Communication	Influences through writing	Company-wide ADRs, blog posts
Leadership	Technical leadership without authority	Leading working groups

Calibration Process

Calibration Meeting Structure

Phase	Duration	Activity
Prep	1 week	Managers prepare packets with evidence
Self-nomination	1 week	Engineers submit promotion packets
Manager discussion	2 hours	Discuss each candidate, calibrate across teams
VP review	1 hour	Cross-org calibration, final decisions
Communication	1 week	Individual feedback sessions

Calibration Guidelines

- **Evidence-based:** Promotions based on demonstrated impact, not tenure
- **Cross-team calibration:** Ensure consistency across managers
- **Bias check:** Review for demographic patterns in promotions
- **Async-friendly:** Written packets enable thoughtful discussion
- **Timely feedback:** Communicate decisions within 2 weeks

Common Anti-Patterns

Anti-Patrón	Consecuencia
"Up or out" culture	Unnecessary pressure, attrition
Manager track as only path	Technical talent leaves
Level inflation	Confusion, compensation issues
No clear criteria	Perception of unfairness
Tenure-based promotions	Mediocrity rewarded
Solo calibration (no cross-check)	Manager bias, inconsistency

Career Development Artifacts

Career Conversation Template

```
## Career Development Conversation

### Where I am now
- Current level: [L4]
- Time at level: [18 months]
- Key accomplishments: [...]
- Areas for growth: [...]

### Where I want to be
- Target level: [L5]
- Timeline: [12 months]
- Key gaps to close: [...]

### How to get there
- Specific projects to take on: [...]
- Skills to develop: [...]
- Mentors to connect with: [...]
- Milestones to track: [...]
```

Individual Development Plan (IDP)

Dimension	Current	Target	Action
Technical depth	Backend strong	Add distributed systems	Take on data pipeline project
Technical breadth	Java only	Polyglot	Learn Go, contribute to service
Leadership	No cross-team	Lead architecture review	Volunteer for ADR process
Communication	Good code reviews	Public speaking	Give 2 tech talks this year

Conexiones

- Mentoring — mentoring supports career growth
- Motivación — growth as intrinsic motivator
- Salud del Equipo — retention as metric
- OKRs — align individual goals with org
- Cultura de Ingeniería — culture shapes career paths
- Diversidad e Inclusión — equitable promotion processes
- Compartir Conocimiento — sharing as promotion evidence

Diversidad Inclusion

El Caso de Negocio

Dimensión	Impacto	Source
Financial	+19% más revenue (McKinsey)	Diversity Wins 2020
Innovation	+19% más innovation revenue	BCG, 2018
Decision making	+87% mejor decision making	Cloverpop
Retention	+22% menor turnover	Deloitte
Employer brand	Top talent attraction	LinkedIn research

Modelos de Diversidad

Tipo	Definición	Ejemplos
Visible	Diferencias percibidas	Género, etnia, discapacidad
Invisible	Diferencias no percibidas	Orientación, neurodiversidad, religión
Experiencial	Background y experiencias	Educación, geografía, industry
Cognitive	Estilos de pensamiento	Analytical vs creative, risk tolerance

Inclusion vs Diversity

- **Diversity** = Who is in the room
- **Inclusion** = Whose voices are heard
- **Belonging** = Who feels they can be themselves

"Diversity is being invited to the party. Inclusion is being asked to dance. Belonging is being able to choose the music." — Vernā Myers

Inclusive Hiring

Job Description Audit

Check	Action
Gendered language	Run through Textio or Gender Decoder
Requirements inflation	Separate "must-have" from "nice-to-have"

Check	Action
Credential bias	Consider skills-based hiring over degree requirements
Culture fit vs culture add	Reframe to "culture contribution"

Structured Interview Process

Phase	Practice
Screening	Blind resume review (remove names, photos)
Phone screen	Standardized questions for all candidates
Technical	Consistent coding challenge across candidates
Behavioral	Structured interview with rubric scoring
Culture	Assess "culture add" not "culture fit"
Debias	Panel interview, score independently before discussing

Interview Scorecard Template

Criterion	1 (Weak)	3 (Meets)	5 (Strong)	Score
Technical skills	Struggles with basics	Completes task	Elegant solution	
Problem solving	Needs heavy hints	Solves independently	Creative approach	
Communication	Unclear explanations	Clear and organized	Inspires confidence	
Collaboration	Works in isolation	Open to input	Elevates team	

ERGs (Employee Resource Groups)

ERG Type	Focus	Activities
Women in Engineering	Gender equity	Mentoring, talks, policy advocacy
LGBTQ+ Tech	Sexual orientation/gender identity	Safe space, events, education
BIPOC in Tech	Racial/ethnic diversity	Mentoring, networking, hiring
Neurodiversity	Different cognitive styles	Workplace accommodations, awareness
Parents in Tech	Work-life balance	Policy feedback, support network

Belonging at Work

The 4 Dimensions of Belonging

Dimension	Description	Signal
Welcomed	People are glad I'm here	Greetings, inclusion in activities
Known	People know my authentic self	Personal conversations, interest in me
Accepted	People accept my differences	No pressure to conform
Supported	People have my back	Advocacy, sponsorship, help

Belonging Practices

Practice	Implementation
Pronouns	Share in Slack profiles, email signatures
Meeting facilitation	Round-robin, async input, no interruption
Social events	Alcohol-free options, varied activities
Documentation	Multiple formats (video, text, visual)
Feedback	Structured, bias-aware feedback processes

Inclusive Meetings

Practice	Why it matters
Pre-read	Async people can prepare and contribute
Round-robin	Ensures everyone speaks, not just loudest
"No interruptions" rule	Protects quieter voices
Written follow-up	Captures ideas from non-verbal contributors
Rotate facilitator	Distributed ownership
Time zone awareness	Fair meeting times across regions

Anti-Patterns

Anti-Patrón	Consecuencia
Performative diversity (optics over substance)	Cynicism, no real change
"Color-blind" approach	Ignores real systemic issues
Burden on minorities to educate	Tokenism, exhaustion
One-time training = done	Checkbox mentality

Anti-Patrón	Consecuencia
Ignoring intersectionality	Overlooks compounded challenges

Conexiones

- Seguridad Psicológica — inclusion requires safety
- Cultura de Ingeniería — inclusive culture
- Motivación — belonging increases motivation
- Contratación — inclusive hiring
- Salud del Equipo — inclusion metrics
- Career Ladders — equitable progression
- Burnout — exclusion contributes to burnout

Burnout Prevencion

Definición (WHO)

La OMS clasifica el burnout como "phenomenon occupational" (no disease) caracterizado por tres dimensiones:

Dimensión	Descripción
Agotamiento	Energía mental y física agotada
Cinismo / Desconexión	Distanciamiento mental del trabajo
Reducida eficacia	Sentimiento de incompetencia y lack de logro

Importante: Burnout es un problema organizacional, no individual. La culpa no es de la persona.

Causas de Burnout en Engineers

Causa organizacional	Ejemplo concreto
Workload insostenible	Deadlines imposibles, crunch persistente
Lack de control	No tener input en decisiones técnicas
Insuficiente reward	Recognition escaso, compensation stagnation
Community breakdown	Silos, lack de support, toxic culture
Absence de fairness	Bias en promotions, decisions opacas
Values mismatch	Trabajo contradice valores personales

Burnout en Tech (Specific)

Factor tech	Por qué causa burnout
Always-on expectations	Slack notifications 24/7, on-call rotas
Rapid context switching	Too many projects, too many tools
Imposter syndrome	Constant learning pressure
Feature factory pressure	Velocity over quality
Incident fatigue	Too many incidents, no root cause fix

Sensores de Detección

Individuales (Self-check)

Señal	Severidad
Difficulty concentrating	🟡 Early
Cynicism about work	🟡 Early
Physical exhaustion	🟠 Medium
Sleep problems	🟠 Medium
Detachment from colleagues	🔴 Advanced
Ineffectiveness at work	🔴 Advanced
Health problems	🔴 Advanced

Organizacionales (Team-level)

Metric	Red flag
eNPS trend	Declining 2+ quarters
Sick days	Increasing trend
Voluntary attrition	Above 15% annually
Overtime hours	Consistently > 10h/week
On-call incidents	> 2 incidents/week/person
Meeting load	> 25h/week

Organizational Prevention Strategies

Workload Management

Strategy	Implementation
Sustainable pace	40h/week standard, no crunch as norm
Capacity planning	80% capacity, 20% buffer
Workload visibility	Team boards showing individual load
Saying no	Empowered to push back on unreasonable deadlines

Autonomy & Control

Strategy	Implementation
Technical decision-making	Engineers choose their tools

Strategy	Implementation
Flexible schedule	Async-first, flexible hours
Project selection	Input into what they work on
Learning time	Dedicated time for growth

Recognition & Growth

Strategy	Implementation
Regular feedback	Quarterly career conversations
Public recognition	Shout-outs in all-hands
Growth opportunities	Stretch projects, conference attendance
Fair compensation	Regular market adjustments

Community Building

Strategy	Implementation
Team rituals	Social time, non-work bonding
Psychological safety	Blameless culture
Mentoring programs	Support networks
ERGs	Community and belonging

Recovery Framework

Individual Recovery

Phase	Duration	Activities
Stabilize	2-4 weeks	Reduce workload, rest, medical support
Reflect	2-4 weeks	Identify root causes, set boundaries
Rebuild	1-3 months	Gradual return, new habits, support system
Sustain	Ongoing	Regular check-ins, boundary maintenance

Organizational Recovery

Action	Owner	Timeline
Root cause analysis	Manager + HR	1-2 weeks

Action	Owner	Timeline
Workload redistribution	Team	Immediate
Process changes	Leadership	1 month
Culture changes	Whole org	3-6 months
Follow-up surveys	HR	Quarterly

Anti-Patterns

Anti-Patrón	Consecuencia
"Just take a vacation"	Treats symptom, not cause
Wellness perks without workload change	Tokenistic, breeds cynicism
Burning out the burnout responders	Helpers need help too
Individual therapy as only solution	Ignores systemic issues
"Hustle culture" glorification	Normalizes unsustainable pace

Conexiones

- Seguridad Psicológica — safety prevents burnout
- Motivación — burnout destroys motivation
- Salud del Equipo — track burnout indicators
- Agile Manifesto — sustainable pace principle
- Diversidad e Inclusión — marginalized groups more affected
- Burnout — self-reference for full content
- Ritmos de Delivery — sustainable delivery cadence
- Scrum — sprint pace impact on burnout

Motivacion Devs

Self-Determination Theory (SDT)

La teoría más influyente en motivación intrínseca (Deci & Ryan).

Los 3 pilares

Pilar	Definición	En el contexto de dev
Autonomy	Sentir control sobre las propias acciones	Elegir tools, how to solve problems
Competence	Sentir que se es bueno en algo	Aprender nuevas tecnologías, resolver challenges
Relatedness	Sentir conexión con otros	Team collaboration, mentoring, communities

Cómo cada pilar afecta la motivación

Baja Autonomy —————> Micromanagement, no input

Alta Autonomy —————> Ownership, empowerment

Baja Competence —————> Imposter syndrome, frustration

Alta Competence —————> Mastery, confidence, flow

Baja Relatedness —————> Isolation, disconnection

Alta Relatedness —————> Team belonging, purpose

Intrinsic vs Extrinsic Motivation

Tipo	Source	Example	Durability
Intrinsic	Internal satisfaction	Solving a hard problem	High
Extrinsic	External rewards	Bonus, promotion	Medium-Low
Introjected	Internalized external	Avoiding shame	Low (anxiety)
Integrated	Aligned with values	Contributing to mission	High

The Overjustification Effect

Cuando se añaden recompensas extrínsecas a una actividad intrínseca:

- La motivación puede disminuir cuando la recompensa se retira
- Mejor: reforzar motivación intrínseca, no reemplazarla

Developer Experience (DX)

El developer experience impacta directamente la motivación.

Las 3 dimensiones del DX

Dimension	Description	Impact
Cognitive load	Mental effort to use tools/processes	Bajo load → más energía para trabajo real
Feedback loops	Tiempo entre acción y resultado	Loops rápidos → más engagement
Flow state	Capacidad de estar "in the zone"	Flow → máxima productividad y satisfacción

Developer Experience Score

Factor	Weight	Preguntas clave
Build system speed	15%	¿Cuánto tarda el build local?
CI/CD reliability	15%	¿Qué % de builds fallan por infra?
Code review turnaround	15%	¿Cuánto tarda en recibirse review?
Documentation quality	15%	¿Puedo encontrar lo que necesito?
Tool consistency	10%	¿Los tools se integran bien?
Onboarding experience	10%	¿Qué tan rápido es productivo un nuevo?
Autonomy level	10%	¿Puedo elegir cómo resolver problemas?
Learning opportunities	10%	¿Aprendo cosas nuevas regularmente?

Flow State en Software

Mihaly Csikszentmihalyi's flow theory aplicada a development.

Conditions for Flow

Condition	Developer context
Clear goals	Well-defined task, acceptance criteria
Immediate feedback	Fast CI, good tests, responsive reviews
Challenge-skill balance	Not too easy (boredom), not too hard (anxiety)
Concentration	Uninterrupted time blocks, no context switching
Sense of control	Ownership over the solution
Loss of self-consciousness	Deep engagement, no performance anxiety

Flow Killers

Killer	Impact	Mitigation
Slack notifications	Breaks concentration	Notification batching
Meetings in the morning	Destroys deep work blocks	Meeting-free mornings
Unreliable tools	Frustration, context switch	Invest in DX
Ambiguous requirements	Constant clarification	Better specs
Multitasking	Reduced quality, slower	Focus on one task

Motivation Anti-Patterns

Anti-Patrón	Consecuencia
Code monkey	No ownership, no meaning
Golden handcuffs	Stay for money, not motivation
Gamification without substance	Superficial engagement
"Pizza parties" as solution	Ignores real issues
Constant pivoting	No sense of completion

Conexiones

- Motivación — self-reference
- Burnout — burnout destroys motivation
- Career Ladders — growth as motivator
- Cultura de Ingeniería — culture enables motivation
- Salud del Equipo — measure motivation
- Agile Manifesto — sustainable pace
- IDEs — good DX = better motivation

Communities Practice

Definición

Una Community of Practice (CoP) es un grupo de personas que comparten una preocupación o pasión por un topic y profundizan su knowledge y expertise interacción regularmente (Wenger & Wenger, 1998).

Elementos fundamentales

Elemento	Description
Domain	El topic compartido (ej: frontend, ML, security)
Community	Las personas que participan activamente
Practice	El cuerpo compartido de knowledge, tools, approaches

CoPs en Software Engineering

Community	Domain	Ejemplo de activities
Frontend Guild	React, CSS, UX	Component library, design system
Backend Guild	APIs, architecture	API guidelines, service patterns
DevOps Community	CI/CD, infra	Pipeline standards, tooling
Security Guild	Application security	Secure coding, threat modeling
Data Engineering	Pipelines, analytics	Data standards, quality practices
Testing Guild	QA, automation	Test strategies, tool selection

CoP vs Chapter vs Guild

Unidad	Scope	Membership	Purpose
Chapter	Within a tribe	Same skill set, same org unit	Best practices, people management
Guild	Cross-tribe	Interest-based, voluntary	Knowledge sharing, standards
CoP	Organization-wide	Voluntary, passion-driven	Deep expertise, innovation

How to Start a CoP

Phase 1: Seed (Month 1)

Activity	Owner	Timeline
Identify passionate advocates	Founder	Week 1
Define domain and value proposition	Founder	Week 1-2
Recruit initial members (5-10)	Founder	Week 2-3
Choose communication channels	Founder	Week 3
Plan first event/meetup	Founder	Week 4

Phase 2: Grow (Months 2-6)

Activity	Cadence
Regular meetings	Quincenal o mensual
Knowledge sharing sessions	Cada meeting
Internal blog/documentation	Semanal
Collaboration on shared artifacts	Ongoing
External speaker invitations	Trimestral

Phase 3: Sustain (Month 6+)

Activity	Purpose
Rotating leadership	Prevent burnout, distribute ownership
Onboarding for new members	Welcome path
Annual planning	Strategic direction
Metrics review	Measure impact
Cross-CoP collaboration	Prevent silos between CoPs

Governance Models

Model	Structure	Best for
Organic	No formal structure, self-organizing	Small orgs, passionate groups
Rotating lead	Leadership rotates quarterly	Medium orgs, preventing single point
Steering committee	3-5 leads make decisions	Large orgs, resource allocation
Sponsored	Leadership sponsors + community leads	Enterprise, needs budget/headcount

Meeting Format Template

Phase	Duration	Activity
Check-in	5 min	Quick round, how everyone is doing
Main topic	30-40 min	Tech talk, demo, or discussion
Open floor	15 min	Questions, sharing, announcements
Action items	5 min	Next steps, follow-ups

Success Metrics

Metric	Target	How to measure
Active members	> 20 per CoP	Attendance tracking
Meeting attendance	> 60% average	Sign-in / Zoom data
Knowledge artifacts	> 1/month	Published docs, tools
Cross-team adoption	Growing usage	Tool/library adoption metrics
Member satisfaction	> 4.0/5	Quarterly survey
New member onboarding	< 2 weeks to first contribution	Time tracking

Anti-Patterns

Anti-Patrón	Consecuencia
Mandatory attendance	Kills intrinsic motivation
No clear domain	Confusion, low engagement
Founder dependency	Single point of failure
Talk-only, no artifacts	Knowledge doesn't persist
Ivory tower	Disconnected from real work
Too many CoPs	Fragmentation, fatigue

Conexiones

- Compartir Conocimiento — CoPs as knowledge sharing vehicle
- Cultura de Ingeniería — CoPs reinforce culture
- Innovation Time — CoPs as innovation context
- Mentoring — peer learning in CoPs
- Trabajo Remoto — async CoP practices

- DevOps — communities as DevOps enabler
- Salud del Equipo — community health metrics

Innovation Time

Modelos de Innovation Time

Modelo	Structure	Ejemplo
20% Time	1 día/semana para projects personales	Google, Atlassian
Hack Days	Días dedicados trimestrales	Airbnb, LinkedIn
Hackathons	Eventos de 24-48h	GitHub, Facebook
Innovation Sprints	1-2 weeks dedicados	Spotify, Basecamp
10% Time	4h/semana	More realistic than 20%

Google's 20% Rule (Historical)

- Cada ingeniero podía dedicar 20% de su tiempo a projects personales
- Resultados: Gmail, Google News, AdSense, Google Transit
- Actualmente menos formalizado pero el principio de innovation time persiste
- Key: no requería approval, solo reporting

Designing Innovation Time

Program Structure

Aspect	Best Practice
Time allocation	10-20% of sprint, or 1 dedicated day/month
Selection criteria	Impact + learning potential + alignment with goals
Team formation	Voluntary, cross-functional teams of 2-4
Demo format	End of period, present to org
Follow-through	Path to production for promising projects

Innovation Time Selection Criteria

Criterion	Weight	Description
Learning potential	30%	Does it teach the team something new?
Business relevance	25%	Could this benefit the product/company?
Technical challenge	20%	Is it interesting enough to motivate?

Criterion	Weight	Description
Feasibility	15%	Can we build a prototype in the time?
Team interest	10%	Is the team excited about it?

Application Template

```
# Innovation Time Project Proposal

## Problem Statement
What problem are we solving or exploring?

## Hypothesis
We believe that [solution] will achieve [outcome].

## Approach
How will we prototype/experiment?

## Success Criteria
What does "done" look like for this time box?

## Resources Needed
- People: [who, how many]
- Time: [hours/days]
- Tools: [any special tools needed]
- Data: [what data access]

## Potential Impact
- Technical: [what we learn]
- Business: [potential value]
```

Hackathon Format

Pre-Hackathon (1-2 weeks before)

Activity	Purpose
Theme selection	Give direction, not constraint
Idea submission	Let people pitch projects
Team formation	Match skills and interests
Logistics prep	Space, food, swag, prizes

Hackathon Timeline

Phase	Duration	Activity
Kickoff	30 min	Rules, theme, teams announced
Hacking	24-48h	Build prototype
Demo prep	2h	Prepare 3-min demo
Demos	1-2h	Each team presents
Judging	30 min	Winners announced
Follow-up	1 week	Decide which projects to pursue

Hackathon Judging Criteria

Criterion	Weight	Description
Innovation	30%	Originality and creativity
Impact	25%	Potential business or technical value
Execution	25%	Working prototype, quality of build
Presentation	10%	Clarity of demo and explanation
Learning	10%	What the team learned

Innovation Labs

Aspect	Description
Dedicated team	Small team (3-5) focused on innovation
Budget	Dedicated funding for experiments
Time horizon	3-6 month projects
Output	Prototypes, research papers, new products
Governance	Separate from main product roadmap

When to Use Innovation Labs

Situation	Appropriate?
Explore new market opportunities	
Build proof of concept for new tech	
Replace regular product development	✗

Situation	Appropriate?
Avoid investing in core product	✗
Need breakthrough innovation	(with core product investment)

Measuring Innovation Time ROI

Metric	Target	How to measure
Projects started	> 4/quarter per team	Project tracking
Projects deployed	> 1/quarter	Production deployments
Learning artifacts	> 2/quarter	Tech talks, docs, blog posts
New tools adopted	> 2/year	Tooling changes
Retention impact	Positive correlation	HR data
Innovation pipeline	Growing ideas backlog	Idea management

Anti-Patterns

Anti-Patrón	Consecuencia
"20% time" without real allocation	Feature work always wins
No follow-through on ideas	Demoralizing, wasted effort
Innovation time for "pet projects"	Inequity, resentment
Innovation without business context	Ivory tower, wasted resources
Over-structured, too many rules	Kills creativity
No demo / no celebration	No incentive to participate

Conexiones

- Communities of Practice — CoPs as innovation context
- Motivación — innovation time boosts motivation
- Cultura de Ingeniería — innovation as cultural value
- Velocity — balance innovation with delivery
- Salud del Equipo — innovation engagement metric
- Agile Manifesto — sustainable pace includes innovation
- IA en Software — AI as innovation topic

README

Cultura y Equipo

Cultura de ingeniería, seguridad psicológica, colaboración, career paths, diversidad, bienestar y motivación. El capital humano como ventaja competitiva en organizaciones de software.

Contenido

#	Archivo	Tema	Nivel	
01	[[../11-Cultura-Equipo/01-cultura-ingenieria	Cultura de Ingeniería]]	Westrum, generativa, frameworks de cultura	Intermedio
02	[[../11-Cultura-Equipo/02-psychological-safety	Seguridad Psicológica]]	Amy Edmondson, Project Aristotle,构建 trust	Intermedio
03	[[../11-Cultura-Equipo/03-remote-hybrid	Trabajo Remoto e Híbrido]]	Async communication, best practices, tools	Intermedio
04	[[../11-Cultura-Equipo/04-collaboration-tools	Herramientas de Colaboración]]	Slack, Notion, Linear, async vs sync	Principiante
05	[[../11-Cultura-Equipo/05-knowledge-sharing	Compartir Conocimiento]]	Tech talks, guilds, documentation culture	Intermedio
06	[[../11-Cultura-Equipo/06-mentorship-coaching	Mentoring y Coaching]]	Programs, pair mentoring, reverse mentoring	Intermedio
07	[[../11-Cultura-Equipo/07-career-ladders	Career Ladders]]	IC vs Management tracks, leveling, calibration	Avanzado
08	[[../11-Cultura-Equipo/08-diversidad-inclusion	Diversidad e Inclusión]]	Inclusive hiring, belonging, programs	Intermedio
09	[[../11-Cultura-Equipo/09-burnout-prevencion	Burnout: Prevención]]	Causes, detection, recovery, org strategies	Intermedio
10	[[../11-Cultura-Equipo/10-motivacion-devs	Motivación de Desarrolladores]]	SDT, intrinsic motivation, flow state	Intermedio
11	[[../11-Cultura-Equipo/11-communities-practice	Communities of Practice]]	What, how to start, governance, metrics	Intermedio
12	[[../11-Cultura-Equipo/12-innovation-time	Innovation Time]]	20% time, hackathons, innovation labs	Intermedio

Secciones relacionadas

- 01 — Fundamentos

- 02 — Estructura
- 03 — Procesos
- 06 — Métricas
- 10 — Gestión de Proyectos